

العلاقات - Eloquent Relationships :الدرس 7

المحتويات

- 1. مقدمة عن العلاقات
- 2. One to One
- 3. One to Many
- 4. Many to Many
- 5. Has One Through
- 6. Has Many Through
- 7. Polymorphic Relations
- 8. Many to Many Polymorphic
- 9. Eager Loading
- 10. أمثلة عملية

مقدمة عن العلاقات

ما هي العلاقات؟

ببعضها البعض بطريقة سهلة وواضحة Models تسمح لك بربط Eloquent العلاقات في

User ↔ Posts (One to Many)
User ↔ Profile (One to One)
Post ↔ Tags (Many to Many)

أنواع العلاقات الأساسية

النوع	الوصف	مثال
One to One	علاقة واحد لواحد	User ← Profile
One to Many	علاقة واحد لمتعدد	User ← Posts
Many to Many	علاقة متعدد لمتعدد	Posts ↔ Tags

النوع	الوصف	مثال
Has One Through	واحد عبر جدول آخر	Country → User → Phone
Has Many Through	متعدد عبر جدول آخر	Country → Users → Posts
Polymorphic	علاقة متعددة الأشكال	Comments على Posts/Videos

One to One

المفهوم

علاقة واحد لواحد - كل سجل في جدول يرتبط بسجل واحد فقط في جدول آخر.

مثال: مستخدم واحد له ملف شخصي واحد

```
User (id=1) ↔ Profile (user_id=1)
User (id=2) ↔ Profile (user_id=2)
```

الإعداد

Migration: users

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamps();
});
```

Migration: profiles

```
Schema::create('profiles', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('bio')->nullable();
    $table->string('avatar')->nullable();
    $table->string('website')->nullable();
    $table->timestamps();
});
```

```
});
```

تعريف العلاقة

Model: User.php

```
class User extends Model
{
    public function profile()
    {
        return $this->hasOne(Profile::class);
    }
}
```

Model: Profile.php

```
class Profile extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

الاستخدام

```
// الحصول على الملف الشخصي للمستخدم
$user = User::find(1);
$profile = $user->profile;

echo $profile->bio;
echo $profile->avatar;

// الحصول على المستخدم من الملف الشخصي
$profile = Profile::find(1);
$user = $profile->user;

echo $user->name;
echo $user->email;

// إنشاء ملف شخصي للمستخدم
$user = User::find(1);
$profile = $user->profile()->create([
    'bio' => 'Laravel Developer',
```

```

        'website' => 'https://example.com',
    ]);

    // أو
    $profile = new Profile([
        'bio' => 'Laravel Developer',
        'website' => 'https://example.com',
    ]);
    $user->profile()->save($profile);

    // التحقق من وجود علاقة
    if ($user->profile) {
        echo "Has profile";
    }

```

One to Many

المفهوم

علاقة واحد لمتعدد – كل سجل في جدول يرتبط بعدة سجلات في جدول آخر.

مثال: مستخدم واحد له عدة منشورات

```

User (id=1)  ← Post (user_id=1)
              ← Post (user_id=1)
              ← Post (user_id=1)

```

الإعداد

Migration: posts

```

Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('title');
    $table->text('content');
    $table->timestamps();
});

```

تعريف العلاقة

Model: User.php

```
class User extends Model
{
    public function posts()
    {
        return $this->hasMany(Post::class);
    }
}
```

Model: Post.php

```
class Post extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

الاستخدام

```
// الحصول على جميع منشورات المستخدم
$user = User::find(1);
$posts = $user->posts;

foreach ($posts as $post) {
    echo $post->title;
}

// مع Query Builder
$posts = $user->posts()->where('published', true)->get();
$posts = $user->posts()->latest()->take(5)->get();

// عد المنشورات
$count = $user->posts()->count();

// الحصول على صاحب المنشور
$post = Post::find(1);
$user = $post->user;

echo $user->name;
```

```
// إنشاء منشور للمستخدم
$user = User::find(1);
$post = $user->posts()->create([
    'title' => 'My First Post',
    'content' => 'Content here...',
]);

// أو
$post = new Post([
    'title' => 'My First Post',
    'content' => 'Content here...',
]);
$user->posts()->save($post);

// إنشاء عدة منشورات
$user->posts()->createMany([
    ['title' => 'Post 1', 'content' => 'Content 1'],
    ['title' => 'Post 2', 'content' => 'Content 2'],
]);
```

Many to Many

المفهوم

علاقة متعددة لمتعدد – كل سجل في جدول يرتبط بعدة سجلات في جدول آخر والعكس.

مثال: منشور له عدة وسوم، ووسم واحد يُستخدم في عدة منشورات

```
Post (id=1) ↔ post_tag ↔ Tag (id=1)
Post (id=2) ↔ post_tag ↔ Tag (id=2)
```

الإعداد

Migration: tags

```
Schema::create('tags', function (Blueprint $table) {
    $table->id();
    $table->string('name')->unique();
    $table->string('slug')->unique();
    $table->timestamps();
});
```

```
});
```

Migration: post_tag (Pivot Table)

```
Schema::create('post_tag', function (Blueprint $table) {
    $table->id();
    $table->foreignId('post_id')->constrained()->onDelete('cascade');
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->timestamps();

    $table->unique(['post_id', 'tag_id']);
});
```

تعريف العلاقة

Model: Post.php

```
class Post extends Model
{
    public function tags()
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

Model: Tag.php

```
class Tag extends Model
{
    public function posts()
    {
        return $this->belongsToMany(Post::class);
    }
}
```

الاستخدام

```
// الحصول على جميع وسوم المنشور
$post = Post::find(1);
$tags = $post->tags;

foreach ($tags as $tag) {
    echo $tag->name;
```

```

}

// الحصول على جميع منشورات الوسم
$tag = Tag::find(1);
$posts = $tag->posts;

// إضافة وسم للمنشور
$post = Post::find(1);
$post->tags()->attach(1); // tag_id = 1
$post->tags()->attach([1, 2, 3]); // عدة وسوم

// إزالة وسم
$post->tags()->detach(1);
$post->tags()->detach([1, 2]); // عدة وسوم
$post->tags()->detach(); // إزالة الكل

// مزامنة الوسوم (استبدال)
$post->tags()->sync([1, 2, 3]);

// مزامنة بدون حذف
$post->tags()->syncWithoutDetaching([4, 5]);

// تبديل (toggle)
$post->tags()->toggle([1, 2]); // إضافة أو إزالة

// التحقق من وجود علاقة
if ($post->tags->contains($tag)) {
    echo "Has tag";
}

```

مع بيانات إضافية Pivot Table

Migration: post_tag

```

Schema::create('post_tag', function (Blueprint $table) {
    $table->id();
    $table->foreignId('post_id')->constrained()->onDelete('cascade');
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->integer('order')->default(0);
    $table->timestamps();
});

```

Model: Post.php

```

class Post extends Model

```



```
{
    public function tags()
    {
        return $this->belongsToMany(Tag::class)
            ->withPivot('order')
            ->withTimestamps();
    }
}
```

الاستخدام:

```
// إضافة مع بيانات إضافية
$post->tags()->attach(1, ['order' => 1]);

// الوصول للبيانات الإضافية
foreach ($post->tags as $tag) {
    echo $tag->name;
    echo $tag->pivot->order;
    echo $tag->pivot->created_at;
}

// مزامنة مع بيانات إضافية
$post->tags()->sync([
    1 => ['order' => 1],
    2 => ['order' => 2],
    3 => ['order' => 3],
]);
```

Has One Through

المفهوم

علاقة واحد عبر جدول آخر – الوصول لعلاقة عبر جدول وسيط.

مثال: دولة ٤ مستخدم ٤ رقم هاتف

```
Country (id=1) → User (country_id=1) → Phone (user_id=1)
```

الإعداد

Migrations:

```
// countries
Schema::create('countries', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->timestamps();
});

// users
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->foreignId('country_id')->constrained();
    $table->string('name');
    $table->timestamps();
});

// phones
Schema::create('phones', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained();
    $table->string('number');
    $table->timestamps();
});
```

تعريف العلاقة

Model: Country.php

```
class Country extends Model
{
    public function phone()
    {
        return $this->hasOneThrough(
            Phone::class, // النموذج النهائي
            User::class,   // النموذج الوسيط
            'country_id',  // Foreign key على users
            'user_id',     // Foreign key على phones
            'id',          // Local key على countries
            'id'           // Local key على users
        );
    }
}
```

الاستخدام

```
$country = Country::find(1);
$phone = $country->phone;

echo $phone->number;
```

Has Many Through

المفهوم

علاقة متعدد عبر جدول آخر - الحصول على مجموعة سجلات عبر جدول وسيط

مثال: دولة ^٤ مستخدمون ^٤ منشورات

```
Country (id=1) → Users (country_id=1) → Posts (user_id=...)
```

تعريف العلاقة

Model: Country.php

```
class Country extends Model
{
    public function posts()
    {
        return $this->hasManyThrough(
            Post::class, // النموذج النهائي
            User::class, // النموذج الوسيط
            'country_id', // Foreign key على users
            'user_id',    // Foreign key على posts
            'id',         // Local key على countries
            'id'          // Local key على users
        );
    }
}
```

الاستخدام

```
// الحصول على جميع منشورات مستخدم الدولة
$country = Country::find(1);
$posts = $country->posts;
```

```
foreach ($posts as $post) {
    echo $post->title;
}

// مع Query Builder
$posts = $country->posts()->where('published', true)->get();
$count = $country->posts()->count();
```

Polymorphic Relations

المفهوم

علاقة متعددة الأشكال – نموذج واحد ينتمي لعدة نماذج أخرى.

مثال: التعليقات على المنشورات والفيديوهات

```
Post (id=1)    → Comment (commentable_type='Post', commentable_id=1)
Video (id=1)   → Comment (commentable_type='Video', commentable_id=1)
```

الإعداد

Migration: comments

```
Schema::create('comments', function (Blueprint $table) {
    $table->id();
    $table->text('content');
    $table->morphs('commentable'); // يضيف commentable_id و commentable_type
    $table->timestamps();
});

// أو يدوياً:
Schema::create('comments', function (Blueprint $table) {
    $table->id();
    $table->text('content');
    $table->unsignedBigInteger('commentable_id');
    $table->string('commentable_type');
    $table->timestamps();

    $table->index(['commentable_id', 'commentable_type']);
});
```

تعريف العلاقة

Model: Comment.php

```
class Comment extends Model
{
    public function commentable()
    {
        return $this->morphTo();
    }
}
```

Model: Post.php

```
class Post extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}
```

Model: Video.php

```
class Video extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}
```

الاستخدام

```
// الحصول على تعليقات المنشور
$post = Post::find(1);
$comments = $post->comments;

// الحصول على تعليقات الفيديو
$video = Video::find(1);
$comments = $video->comments;

// الحصول على العنصر من التعليق
```

```

$comment = Comment::find(1);
$commentable = $comment->commentable;

if ($commentable instanceof Post) {
    echo "Comment on post: " . $commentable->title;
} elseif ($commentable instanceof Video) {
    echo "Comment on video: " . $commentable->title;
}

// إنشاء تعليق
$post = Post::find(1);
$comment = $post->comments()->create([
    'content' => 'Great post!',
]);

$video = Video::find(1);
$comment = $video->comments()->create([
    'content' => 'Nice video!',
]);

```

Many to Many Polymorphic

المفهوم

Many to Many و Polymorphic علاقة متعدد لمتعدد متعددة الأشكال – علاقة معقدة تجمع بين

مثال: وسوم على المنشورات والفيديوهات

```

Post (id=1) ↔ taggables ↔ Tag (id=1)
Video (id=1) ↔ taggables ↔ Tag (id=1)

```

الإعداد

Migration: taggables

```

Schema::create('taggables', function (Blueprint $table) {
    $table->id();
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->morphs('taggable'); // taggable_id و taggable_type
    $table->timestamps();
});

```

```
});
```

تعريف العلاقة

Model: Tag.php

```
class Tag extends Model
{
    public function posts()
    {
        return $this->morphedByMany(Post::class, 'taggable');
    }

    public function videos()
    {
        return $this->morphedByMany(Video::class, 'taggable');
    }
}
```

Model: Post.php

```
class Post extends Model
{
    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');
    }
}
```

Model: Video.php

```
class Video extends Model
{
    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');
    }
}
```

الاستخدام

```
// إضافة وسوم للمنشور
$post = Post::find(1);
```

```
$post->tags()->attach([1, 2, 3]);

// إضافة وسوم للفيديو
$video = Video::find(1);
$video->tags()->attach([1, 2, 3]);

// الحصول على جميع منشورات الوسم
$tag = Tag::find(1);
$posts = $tag->posts;
$videos = $tag->videos;

// مزامنة
$post->tags()->sync([1, 2, 3]);
```

Eager Loading

المشكلة: N+1 Query

```
// سيء: N+1 queries
$posts = Post::all(); // 1 query

foreach ($posts as $post) {
    echo $post->user->name; // N queries (واحد لكل منشور)
}
// المجموع: 1 + N queries
```

الحل: Eager Loading

```
// جيد: فقط 2 queries
$posts = Post::with('user')->get(); // 2 queries فقط

foreach ($posts as $post) {
    echo $post->user->name; // إضافية queries لا توجد
}
```

Eager Loading أمثلة على

```
// تحميل علاقة واحدة
$posts = Post::with('user')->get();
```



```
// تحميل عدة علاقات
$post = Post::with(['user', 'comments'])->get();

// تحميل علاقات متداخلة
$post = Post::with('user.profile')->get();
$post = Post::with('comments.user')->get();

// تحميل علاقات متعددة متداخلة
$post = Post::with([
    'user',
    'comments.user',
    'tags'
])->get();

// تحميل مع شروط
$post = Post::with(['comments' => function ($query) {
    $query->where('approved', true)
        ->orderBy('created_at', 'desc');
}])->get();

// تحميل فقط عدد معين
$post = Post::with(['comments' => function ($query) {
    $query->latest()->take(5);
}])->get();

// Lazy Eager Loading
$post = Post::all();
$post->load('user'); // تحميل بعد الاسترجاع

// تحميل إذا لم يكن محملاً
$post->loadMissing('user');
```

عدد العلاقات - withCount

```
// عدد التعليقات
$post = Post::withCount('comments')->get();

foreach ($post as $post) {
    echo $post->comments_count; // عدد التعليقات
}

// عدد عدة علاقات
$post = Post::withCount(['comments', 'likes'])->get();

// عدد مع شروط
$post = Post::withCount([
```

```
'comments',
'comments as approved_comments_count' => function ($query) {
    $query->where('approved', true);
}
])->get();
```

أمثلة عملية

مثال 1: نظام مدونة كامل

```
// المستخدم وملفه الشخصي
$user = User::with('profile')->find(1);
echo $user->name;
echo $user->profile->bio;

// منشورات المستخدم مع التعليقات
$user = User::with('posts.comments')->find(1);

foreach ($user->posts as $post) {
    echo $post->title;
    echo "Comments: " . $post->comments->count();
}

// منشور مع صاحبه والتعليقات والوسوم
$post = Post::with(['user.profile', 'comments.user', 'tags'])
->find(1);

echo $post->title;
echo "By: " . $post->user->name;
echo "Bio: " . $post->user->profile->bio;

foreach ($post->comments as $comment) {
    echo $comment->user->name . ": " . $comment->content;
}

foreach ($post->tags as $tag) {
    echo $tag->name;
}
```

مثال 2: إحصائيات

```
// مستخدمون مع عدد المنشورات والتعليقات
$users = User::withCount(['posts', 'comments'])
    ->orderBy('posts_count', 'desc')
    ->get();

foreach ($users as $user) {
    echo "{$user->name}: {$user->posts_count} posts, {$user->comments_count}
    comments";
}

// منشورات مع عدد التعليقات المعتمدة
$posts = Post::withCount([
    'comments as approved_comments_count' => function ($query) {
        $query->where('approved', true);
    }
])->get();
```

مثال 3: البحث في العلاقات

```
// منشورات لمستخدمين من دولة معينة
$posts = Post::whereHas('user', function ($query) {
    $query->where('country_id', 1);
})->get();

// منشورات لها تعليق من مستخدم معين
$posts = Post::whereHas('comments', function ($query) {
    $query->where('user_id', 1);
})->get();

// منشورات ليس لها تعليقات
$posts = Post::doesntHave('comments')->get();

// منشورات لها على الأقل 5 تعليقات
$posts = Post::has('comments', '>=', 5)->get();
```

نصائح مهمة

أفضل الممارسات ؟

1. Eager Loading: استخدم



```
// جيد  
$posts = Post::with('user')->get();
```

```
// سيء (N+1 problem) $posts = Post::all(); foreach ($posts as $post) { echo  
$post->user->name; }
```

```
2. **لإحصائيات withCount استخدم**  
```php  
// جيد
$posts = Post::withCount('comments')->get();

// سيء
$posts = Post::all();
foreach ($posts as $post) {
 $count = $post->comments()->count(); // query لكل منشور
}
```

### 3. حدد الأعمدة المطلوبة:

```
$posts = Post::with('user:id,name,email')->get();
```

### 4. join بدلاً من whereHas استخدم:

```
// أوضح وأسهل
$posts = Post::whereHas('tags', function ($query) {
 $query->where('name', 'Laravel');
})->get();
```

## ❗❗ أخطاء شائعة

### 1. Eager Loading نسيان:

```
// مشكلة N+1
$posts = Post::all();
foreach ($posts as $post) {
 echo $post->user->name; // query لكل منشور
```

```
}
```

## 2. مرتين استخدام `get()`:

```
// خطأ
$posts = Post::with('user')->get()->get();
```

```
// صحيح $posts = Post::with('user')->get();
```

```
3. ** نسيان تعريف belongsTo:**
```php  
// فقط hasMany  
class User extends Model {  
    public function posts() {  
        return $this->hasMany(Post::class);  
    }  
}  
  
// belongsTo في Post مع  
class Post extends Model {  
    public function user() {  
        return $this->belongsTo(User::class);  
    }  
}
```

الخطوة التالية

بعد إتمام هذا الدرس، أنت الآن جاهز لـ

8 الدرس: Validation and Form Requests

- Basic Validation
- Form Request Classes
- Custom Validation Rules
- Error Messages

؟! تعلم سعيد